Project 3: Zero-Knowledge and Non-Interactive Arguments

1 Introduction

In this project, you'll extend your implementation of the Delphian succinct argument from Project 2 by adding two crucial properties: zero-knowledge and non-interactivity. The resulting protocol, zkDelphian, is a zkSNARK for R1CS. We will give you starter code for the major pieces: zero-knowledge versions of sumcheck, Quokka, and the Delphian protocol itself. These will be described in Section 4.2, Section 4.3, and Section 4.4 respectively. You will also need to implement four auxiliary Σ-protocols (described in Section 4.1) from starter code. We will give you a general-purpose library for Fiat-Shamir—described in Section 3—but you must correctly apply it to make zkDelphian non-interactive.

2 Modifications to P1 and P2

This project builds directly on top of your Project 1 and Project 2 code. Before you begin, you will need to make some changes to those crates. In particular, your P1 types need to implement `Serialize`/`Deserialize` (so they can be absorbed into the transcript) and a new `FromBytes` trait (so they can be squeezed out as challenges), and your P2 types similarly need serialization support.

The starter code repository contains a file `p3_migration.md` with the full list of required changes. You have two options:

1. **Run the automated script.** From the directory containing your `p1/`, `p2/`, and `p3/` folders, run `python ./p3_migration .`. The script will create a backup of your existing code and print a diff of every change so you can verify what it did.

2. **Apply the changes by hand.** The migration guide lists each file and the exact modifications needed.

We recommend starting with the script and falling back to manual changes if anything looks off. You can confirm that everything is working by (from within the `p3` directory) running `cargo check`. You will likely see plenty of warnings about unused variables etc, but there should not be any errors.

3 The Transcript struct

In Project 2, your protocols were interactive; the verifier sampled random challenges and sent them to the prover directly. Now that we are applying the Fiat-Shamir transform, those random challenges must be derived from the protocol messages by hashing. A naive approach is to manually concatenate values and feed them into a hash function at each step, but this is surprisingly easy to get wrong, particularly when composing multiple protocols together. The `Transcript` struct, which we provide for you in this project, is designed to make Fiat-Shamir both easier to use and harder to misuse. Its design is inspired by the Merlin transcript

library. Though you will not be required to implement anything in `src/transcript.rs`, we nonetheless encourage you to read this implementation and understand what is going on.

Basic usage. A `Transcript` wraps an internal hash state (a Keccak sponge). You interact with it through two operations:

- `append_message(label, value)`: absorb a labeled value into the running hash state. The value is serialized using BCS (Binary Canonical Serialization) via Serde's `Serialize` trait, which guarantees that distinct values produce distinct byte representations. Any type that implements `Serialize` can be appended.
- `get_challenge(label)`: squeeze out a pseudorandom challenge that depends on everything absorbed so far. The raw bytes from the sponge are converted to the desired type via a `FromBytes` trait (see the migration section). Any type that implements `FromBytes` can be squeezed out as a challenge.

Both the prover and the verifier create a `Transcript` with the same protocol label and then perform the *exact same sequence* of `append_message` and `get_challenge` calls. Because the sponge is deterministic, they derive identical challenges as long as the absorbed messages match.

For example, suppose a prover sends a commitment C and then needs a random challenge c from the verifier. In the non-interactive setting, both sides would write:

```
let mut transcript = Transcript::new("my_protocol");
transcript.append_message("first_commitment", &C);
let c: Scalar = transcript.get_challenge("challenge");
```

The prover runs this code to *derive* the challenge; the verifier runs the same code to *recompute* it and check consistency. Note the type annotation on `get_challenge`: because the return type is generic (any type implementing `FromBytes`), you will often need to specify it explicitly, though Rust can sometimes infer it from context.

Why the transcript abstraction?. Consider a protocol with two rounds. A first attempt at Fiat-Shamir might look like:

```
let c1 = hash(C1);                // first challenge
let c2 = hash(C1 || C2 || z1);    // second challenge
```

This is workable for a toy example, but in a real proof system like the one you are building—where sumcheck, polynomial commitments, and Σ-protocols are all composed together—the manual approach becomes a minefield. Let's walk through what can go wrong.

Ambiguous encoding. Suppose you compute a challenge by concatenating two byte strings: `hash(a || b)`. If `a` and `b` don't have fixed lengths, then the inputs `a = "AB", b = "CD"` and `a = "ABC", b = "D"` produce the same hash input `"ABCD"`. A malicious prover could exploit this to substitute one pair of messages for another without changing the challenge. The transcript prevents this by framing every `append_message` call with a domain-separator tag and an explicit length prefix, so distinct inputs always produce distinct absorptions. (One caveat: BCS does not include type tags in its encoding, so it is theoretically possible for a value of one type to have the same serialization as a value of a different type. In practice this is not a concern, because the label on each `append_message` call identifies the role of the data, and at any given point in the protocol the type of the expected message is generally fixed by the protocol specification, so a prover cannot substitute e.g. a group element where a scalar is expected.)

Missing bindings. Forgetting to hash in a value (say, a commitment or the statement being proved) means the resulting challenge is not bound to it, potentially allowing a prover to cheat. With manual hashing in a multi-round, multi-component protocol, you have to carefully track which values have been included in which hash calls, and it is easy to accidentally leave one out. Routing everything through a single `Transcript` makes omissions easier to spot in review: if a value doesn't appear in an `append_message` call, it isn't bound. That said, the transcript alone does not fully solve this problem; it still relies on the implementor to remember to absorb every relevant value. Trail of Bits' `decree` library takes this further by requiring protocols to declare up front which values must be absorbed before a challenge can be extracted; if any are missing, the library refuses to produce the challenge. See their blog post for more on the design.

A particularly important instance of this: the *public parameters and statement* of the protocol must be absorbed into the transcript before any proof messages are processed. If they are not, the challenges are not bound to what is actually being proved, and a proof generated for one statement could verify against a different one. In your implementation, this means that the first thing a prover or verifier should do is append the public parameters and the instance to the transcript.

Cross-protocol replay and composition. This is perhaps the subtlest issue, and the one that most motivates the transcript design. Your final argument is not a single monolithic protocol. Rather, it is built by composing sumcheck, a polynomial commitment scheme, and several Σ-protocols. You'll notice that all of the proving and verifying functions accept a `&mut Transcript` parameter rather than creating their own. This means a single transcript is threaded through the entire protocol, so every sub-protocol's challenges depend on the full history of messages from *all* preceding sub-protocols.

Why does this matter? Without a shared transcript, each sub-protocol's challenges would be derived independently. A malicious prover could then take a legitimately generated sub-proof from one context and replay it in a different context where the surrounding messages are different. With a shared transcript, this is impossible: the challenges in, say, a Σ-protocol that runs after sumcheck depend on all of the sumcheck messages, so a proof generated under a different sumcheck execution would produce completely different challenges and fail verification.

The protocol label passed to `Transcript::new` provides the outermost layer of domain separation, ensuring that transcripts from entirely different protocols (e.g. `Transcript::new("zk_delphian")` vs. `Transcript::new("some_other_protocol")`) diverge immediately, even before any messages are absorbed. Real-world deployments would also typically include version strings in these labels, to prevent malleability across versions. Within the protocol, threading a single `&mut Transcript` through sub-protocols provides the compositional binding.

Passing transcripts as parameters. This compositional story is the reason that proof functions in the starter code accept a `&mut Transcript` rather than creating one internally. This is a deliberate API convention (borrowed from Merlin): *proof implementations should never create their own transcript; they should always receive one from the caller.*

This convention has two benefits. First, it is what makes sequential composition work. If `sumcheck_prove` and `sigma_prove` each created a fresh transcript, there would be no way to bind their challenges together. You would have to manually figure out how to splice the hash states, which is exactly the kind of error-prone bookkeeping the transcript is meant to eliminate. By accepting a `&mut Transcript`, each function simply continues absorbing into whatever state the caller has built up so far, and composition happens automatically.

Second, it lets the caller bind additional context *before* invoking the proof. For example, suppose you have a generic Schnorr proof function. By appending a message to the transcript before calling it, you can turn the same proof function into a signature scheme: the challenge will now depend on the signed message, not just

the proof's own commitments. More generally, the caller is in the best position to know what application-level context should be bound to the proof, and the `&mut Transcript` parameter gives it a natural place to do so.

How labels should be chosen. Every call to `append_message` and `get_challenge` takes a *label* string. These labels serve as lightweight domain separators *within* a protocol. Two principles guide their selection:

1. **Labels should describe the role of the data rather than its type.** Use `"round_1_commitment"` rather than `"group_element"`; use `"claimed_sum"` rather than `"scalar"`. If your protocol sends two commitments in the same round, give them different labels (e.g. `"comm_left"` and `"comm_right"`).

2. **Labels should be fixed strings, not runtime data.** Anything that varies between executions (an index, a counter, a user-supplied identifier) belongs in the *message body*, not the label. The labels are part of the protocol definition; the messages are the data that changes from proof to proof.

Not every label needs to be globally unique across all protocols; however, the first label you write to the transcript inside a protocol should uniquely identify that protocol.

4 Overview of zkDelphian

Before we start, it will be useful to recall our Delphian protocol and think at a high level about what we're trying to accomplish. The goal of ZK, recall, is to *hide information about the witness*. We formalize this guarantee using the simulation-based definition from class, but this is just a way of making the basic idea precise. Now, given this, it's useful to ask: *where in the Delphian protocol can the verifier learn information about the witness*?

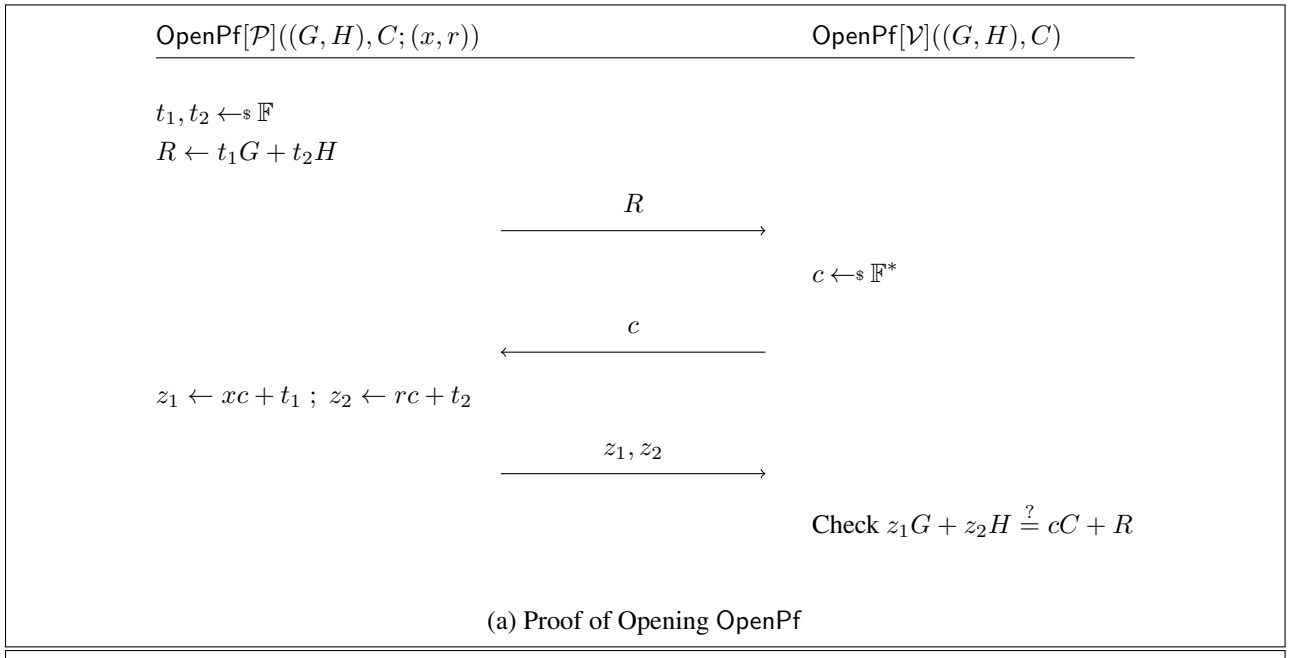
Sumcheck takes a polynomial derived from the witness as input, and as discussed in class, the round polynomials reveal information about the prover's polynomial to the verifier. We therefore need a zero-knowledge version of sumcheck. In class we discussed one approach; here we present a more optimized construction that compresses all of the verifier's round checks into a single inner-product proof.

Quokka reveals witness information in two ways. First, the commitment itself from P2 is not hiding—it is a deterministic function of the polynomial—so we need to add a random blinding factor to make it hiding. Second, the opening proof leaks information: in plain Quokka, the prover sends $\vec{b}^\top M$ to the verifier, where $\vec{b}$ is a vector derived from a verifier-chosen evaluation point. This reveals partial information about M (exercise: characterize exactly what is revealed). Instead of having the prover send $\vec{b}^\top M$ and the verifier check correctness by recomputing C', we use a dot-product proof on the derived commitment C'.

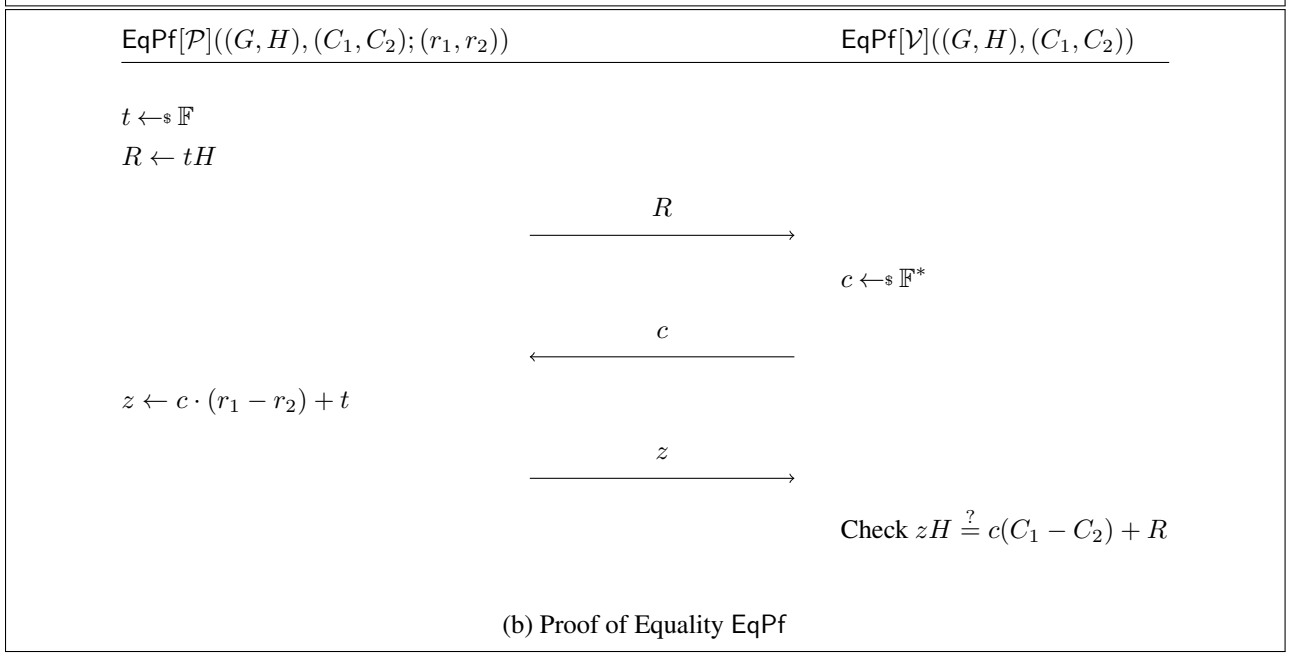
Finally, Delphian itself reveals some information about the witness in the transition between the first and second phase. In P2's Delphian, the prover gave the verifier values v_A, v_B, v_C that the verifier used as inputs to the second-phase sumchecks; the verifier also checked that $(v_A v_B - v_C)\widetilde{\mathsf{eq}}(r, r') = H'$, where H' is claimed to be the evaluation of $h(x)$ at a random point. To make this step zero-knowledge, the prover instead sends Pedersen commitments to v_A, v_B, v_C and uses Σ-protocols to prove the claimed relationship holds under the commitments. These commitments then serve as inputs to the second-phase zero-knowledge sumchecks, standing in for the committed evaluations of the p_M polynomials.

4.1 A Σ-Protocol Toolbox

To make sumcheck, Quokka, and Delphian zero-knowledge, we will use auxiliary Σ-protocols for specific tasks. We describe and discuss these first.



(a) Proof of Opening OpenPf



(b) Proof of Equality EqPf

Figure 1: Two Σ-protocols: OpenPf for proving knowledge of a Pedersen commitment opening and EqPf for proving equality of two committed values.

Opening proof. The first Σ-protocol we need is one we discussed in class: Okamoto's protocol for proving knowledge of the opening of a Pedersen commitment. The exact relation is

$$\mathcal{R}_{\mathsf{Open}} = \left\{((G,H),C;(x,r)) : C = xG + rH\right\} \ .$$

The protocol OpenPf is reproduced in Figure 1a.

Equality proof. The second Σ-protocol we will use lets us prove that two Pedersen commitments are commitments of the same value under different randomness. The precise relation is

$$\mathcal{R}_{\mathsf{Eq}} = \left\{((G,H),(C_1,C_2);(r)) : C_1 = C_2 + rH\right\} \ .$$

At first glance, this relation does not seem to exactly prove equality of the two committed values. However, note that as long as the discrete logarithm of G to the base H is unknown, the difference between C_1 and C_2 can be written as some scalar times H only if the G-components of C_1 and C_2 are equal. The protocol EqPf is in Figure 1b.

We note that the equality proof EqPf only proves that the two commitments are to the same value, but does not automatically allow extracting the value. Looking ahead, in zkDelphian (Figure 5) it is thus necessary to also invoke OpenPf during the transition from phase 1 to phase 2. This is the only way to guarantee that the value v_C can be extracted from its commitment C_{v_C}, which is necessary for knowledge soundness.

Product proof. The third Σ-protocol we use lets us prove that the product of two committed values is equal to a third committed value. The precise relation is

$$\mathcal{R}_{\mathsf{Prod}} = \left\{ \begin{array}{c} ((G,H),(X,Y,Z);(x,y,r_x,r_y,r_z)) : \\ X = xG + r_xH,\ Y = yG + r_yH,\ Z = (xy)G + r_zH \end{array} \right\} .$$

The protocol ProdPf is reproduced in Figure 2.

Dot product proof. The fourth Σ-protocol we need is also one we discussed in class, but in a different guise. In class we described this as an opening protocol for a Pedersen-committed polynomial; here we will use it as a more general proof of a dot product relationship between a committed vector and a public vector. The precise relation is

$$\mathcal{R}_{\mathsf{DotProd}} = \left\{ \begin{array}{c} ((n,\vec{G},G,H),(C_{\vec{x}},C_v,\vec{a});(\vec{x},v,r_{\vec{x}},r_y)) : \\ C_{\vec{x}} = \mathsf{msm}(\vec{x},\vec{G}) + r_{\vec{x}}H,\ C_v = vG + r_yH,\ v = \langle \vec{x},\vec{a}\rangle \end{array} \right\} .$$

The protocol DotProdPf is reproduced in Figure 2.

Compact proofs via the Fiat-Shamir trick. The natural way to apply Fiat-Shamir to a Σ-protocol is to include the prover's first message (e.g., R in OpenPf) in the proof, absorb it into the transcript, and derive the challenge c. This works, but R is a group element—typically the most expensive object to transmit.

We can do better. The prover proceeds exactly as in the interactive protocol—computing R, deriving c from the transcript, and computing the response $\vec{z}$—but it includes $(c,\vec{z})$ in the proof instead of $(R,\vec{z})$. This makes the proof smaller since c is a field element and much smaller than a group element. The verifier, on the other hand, operates in a different order: it first rearranges the verification equation to recover R from c and $\vec{z}$ (for example, in OpenPf it computes $R = z_1G + z_2H - cC$), then absorbs the reconstructed R into the transcript, squeezes out a challenge c', and checks that $c' = c$.

In your implementation, OpenPf, EqPf, and ProdPf should all use this trick: the proof struct stores $(c,\vec{z})$ rather than $(R,\vec{z})$. The DotProdPf protocol uses the same idea—the proof stores e along with the scalar responses $(\vec{x}',r_{\vec{x}'},r_{v'})$ rather than the two first-move group elements (C_1,C_2)—and the verifier reconstructs both C_1 and C_2 before re-deriving the challenge.

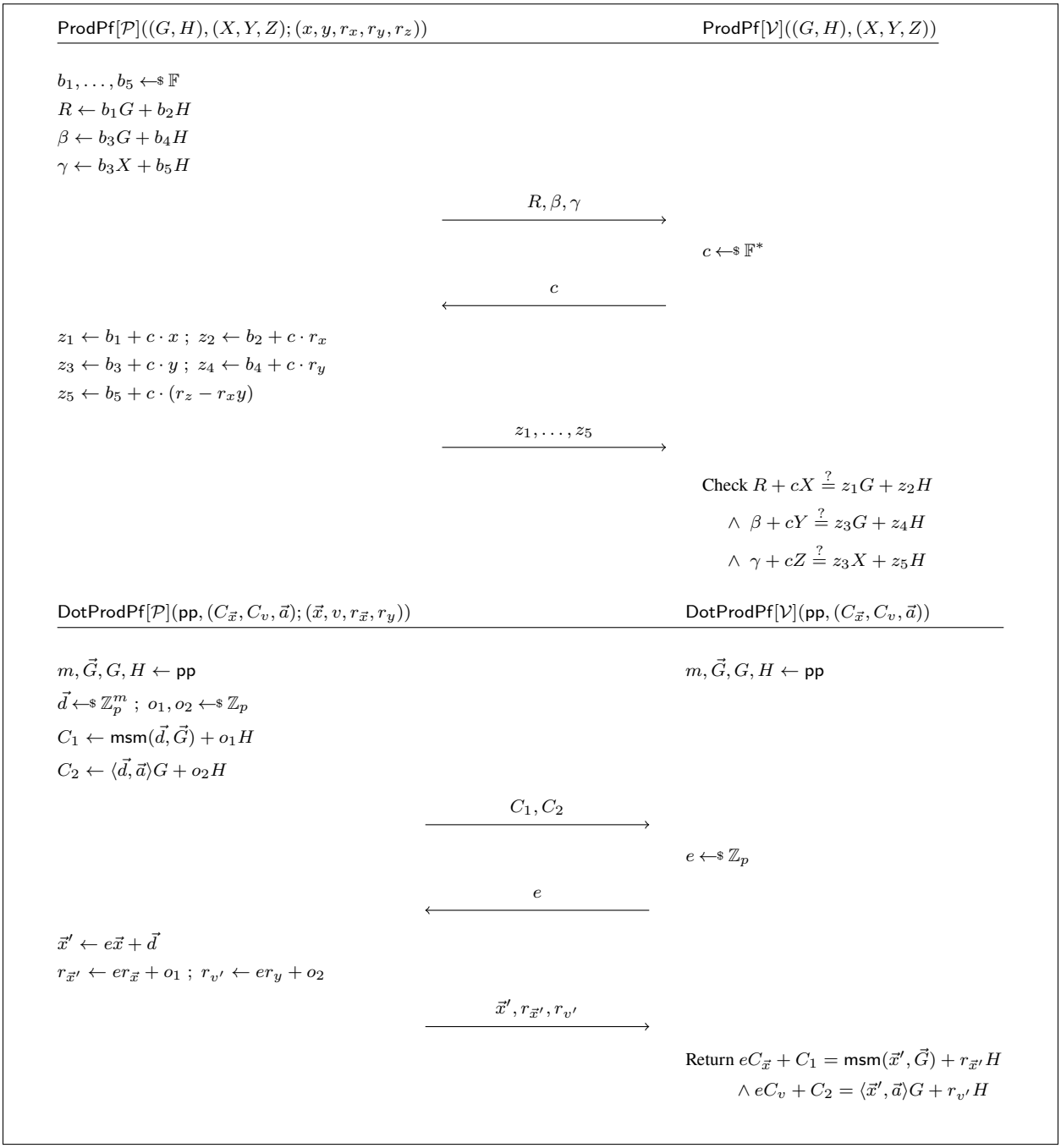


Figure 2: The ProdPf and DotProdPf Σ-protocols. ProdPf proves that the product of two committed values equals a third committed value. DotProdPf proves that the dot product of a committed vector with a public vector equals a committed scalar.

4.2 Zero-Knowledge Sumcheck

The pseudocode of the ZK sumcheck is in Figure 3. As it is used in zkDelphian, we again need this to be the reduction version of the sumcheck, except with the zero-knowledge security guarantee for the claimed evaluations. Thus, the verifier only takes a commitment C_v, to the claimed sum v of the evaluations of the prover's polynomial over the hypercube.

The per-round check. Consider what a straightforward zero-knowledge sumcheck might look like. Each round, the prover commits to the round polynomial's coefficients, say using separate scalar Pedersen commitments to $\pi_{j,0}, \ldots, \pi_{j,d}$ (where $g_j(X) = \sum_{i=0}^{d} \pi_{j,i} X^i$). The verifier's consistency check in round j is $g_j(0) + g_j(1) = g_{j-1}(r_{j-1})$. Expanding both sides:

$$\underbrace{2\pi_{j,0} + \pi_{j,1} + \cdots + \pi_{j,d}}_{g_j(0)+g_j(1)} \;=\; \underbrace{\pi_{j-1,0} + \pi_{j-1,1}\, r_{j-1} + \cdots + \pi_{j-1,d}\, r_{j-1}^{d}}_{g_{j-1}(r_{j-1})}.$$

This is a linear equation in the committed coefficients. Since Pedersen commitments are homomorphic, the verifier can compute a commitment to each side and use EqPf to check they match. The same is true of the base-case check ($g_1(0) + g_1(1) = v$) and the final evaluation check ($g_v(r_v) = g(\vec{r})$) — each is linear. So this straightforward approach works, but at the cost of $v{+}1$ sigma-protocol invocations. Perhaps surprisingly, we can compress all checks into a *single* DotProdPf (Section 4.1).

The checks as a matrix equation. Concatenate the coefficient vectors of all round polynomials into a single vector $\vec{\pi} = (\pi_{1,0}, \ldots, \pi_{1,d}, \ldots, \pi_{v,0}, \ldots, \pi_{v,d})$. Since every check is a linear equation in $\vec{\pi}$, the entire system can be written as $M\vec{\pi} = \vec{s}$, where M is $(v{+}1) \times v(d{+}1)$ and $\vec{s} = (v,\, 0,\, \ldots,\, 0,\, g(\vec{r}))^{\top}$. For example, for $v = 3$ and $d = 2$, we can write the verifier's checks as:

$$\underbrace{\begin{pmatrix} 2 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ -1 & -r_1 & -r_1^2 & 2 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 & -r_2 & -r_2^2 & 2 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & r_3 & r_3^2 \end{pmatrix}}_{M} \underbrace{\begin{pmatrix} \pi_{1,0} \\ \vdots \\ \pi_{1,2} \\ \pi_{2,0} \\ \vdots \\ \pi_{2,2} \\ \pi_{3,0} \\ \vdots \\ \pi_{3,2} \end{pmatrix}}_{\vec{\pi}} = \underbrace{\begin{pmatrix} v \\ 0 \\ 0 \\ g(\vec{r}) \end{pmatrix}}_{\vec{s}}.$$

Row 0 is the base case: $(2, 1, \ldots, 1)$ computes $g_1(0) + g_1(1)$. Each middle row links two consecutive rounds: $(-1, -r_j, \ldots, -r_j^d)$ evaluates the previous round polynomial at r_j, and the $(2, 1, \ldots, 1)$ block computes $g_{j+1}(0) + g_{j+1}(1)$; the row equals zero exactly when these two quantities match. The last row is the final evaluation check.

Flattening to a single check (BatchCoeffs). Write M_k for the k-th row of M, so that $\langle M_k, \vec{\pi}\rangle = s_k$ is the k-th check. If we pick random weights $\vec{\rho} = (\rho_0, \ldots, \rho_v)$ and take the weighted sum $\vec{y} = \sum_k \rho_k M_k$, we get a single equation: $\langle \vec{y},\ \vec{\pi}\rangle = \sum_k \rho_k\, s_k = \rho_0 \cdot v + \rho_v \cdot g(\vec{r})$ (the middle entries of $\vec{s}$ are zero, so they drop out).

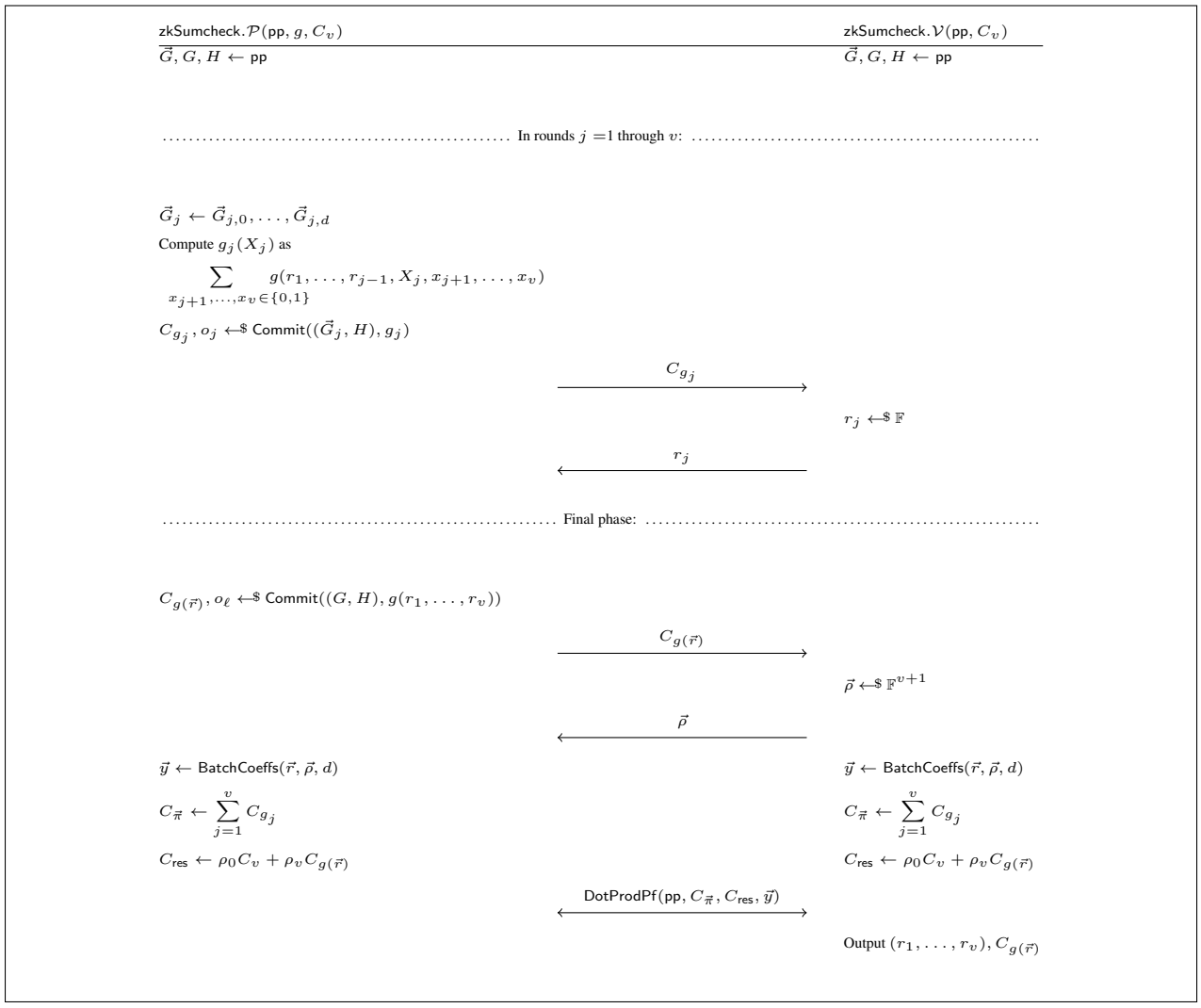


Figure 3: A zero-knowledge sumcheck reduction. The protocol ends with the verifier outputting the evaluation point $(r_1, \ldots, r_v)$ as well as a *commitment* to the claimed evaluation $g(r_1, \ldots, r_v)$.

Clearly this holds if $M\vec{\pi} = \vec{s}$. Moreover, by Schwartz–Zippel, if $M\vec{\pi} \neq \vec{s}$ then this holds with probability at most $v/|\mathbb{F}|$.

This is what $\vec{y} = \mathsf{BatchCoeffs}(\vec{r}, \vec{\rho}, d)$ computes. A single DotProdPf on the equation $\langle \vec{y}, \vec{\pi} \rangle = \rho_0 v + \rho_v g(\vec{r})$ replaces all $v+1$ individual checks. However, we still need to compute the Pedersen vector commitment to $\vec{\pi}$ itself, and doing so is slightly subtle.

Constructing the vector commitment. The prover cannot commit to $\vec{\pi}$ up front, because the round polynomials are computed adaptively: g_j depends on the challenge r_{j-1}, which depends (via Fiat–Shamir) on the commitment to g_{j-1}.

So instead, the prover commits *incrementally*. In each round j, it vector-commits to that round's coefficients $\vec{\pi}_j = (\pi_{j,0}, \ldots, \pi_{j,d})$ using a dedicated chunk of $d+1$ generators $\vec{G}_j$:

$$C_{g_j} = \mathsf{msm}(\vec{\pi}_j,\, \vec{G}_j) + o_j\, H$$

where o_j is a fresh blinding factor. Let $\vec{G} = (\vec{G}_1 \| \cdots \| \vec{G}_v)$ be the full generator vector. Because the chunks $\vec{G}_j$ are disjoint slices of $\vec{G}$, summing the per-round commitments telescopes:

$$\begin{aligned} C_{\vec{\pi}} \;&=\; \sum_{j=1}^{v} C_{g_j} = \sum_{j=1}^{v} \big[\mathsf{msm}(\vec{\pi}_j,\, \vec{G}_j) + o_j\, H\big] \\ &= \mathsf{msm}(\vec{\pi},\, \vec{G}) \;+\; \Big(\sum_{j=1}^{v} o_j\Big) H. \end{aligned} \tag{4.1}$$

The result is a valid Pedersen vector commitment to $\vec{\pi}$ under generators $\vec{G}$, with blinding factor $r_{\vec{\pi}} = \sum_j o_j$. The prover knows $r_{\vec{\pi}}$ because it accumulates the blinding factors as each round completes. The verifier can compute $C_{\vec{\pi}}$ just by summing the round commitments it already received.

The final DotProdPf**.** At this point, both the prover and verifier know $\vec{y}$ and agree that the claim to check is $\langle \vec{y},\, \vec{\pi} \rangle = \rho_0 \cdot v + \rho_v \cdot g(\vec{r})$. Recall from Section 4.1 that DotProdPf proves, for a public vector $\vec{a}$,

$$C_{\vec{x}} = \mathsf{msm}(\vec{x}, \vec{G}) + r_{\vec{x}}\, H, \quad C_{res} = res \cdot G + r_{res}\, H, \quad res = \langle \vec{x},\, \vec{a} \rangle.$$

We instantiate it as follows. Both parties set $\vec{a} \leftarrow \vec{y}$ and compute $C_{\vec{x}} \leftarrow \sum_{j=1}^{v} C_{g_j}$ and $C_{res} \leftarrow \rho_0\, C_v + \rho_v\, C_{g(\vec{r})}$ from public proof data. The prover's witness is $\vec{x} = \vec{\pi}$, $res = \rho_0 v + \rho_v g(\vec{r})$, and the two blinding factors:

- $r_{\vec{x}} = \sum_j o_j$, the accumulated per-round blinding; this matches $C_{\vec{x}}$ by equation (4.1).
- $r_{res} = \rho_0\, r_v + \rho_v\, o_\ell$, where r_v is the blinding in C_v and o_ℓ the blinding in $C_{g(\vec{r})}$. This follows from expanding C_{res} by Pedersen's homomorphism: $C_{res} = \rho_0\, C_v + \rho_v\, C_{g(\vec{r})} = (\rho_0\, v + \rho_v\, g(\vec{r}))\, G + (\rho_0\, r_v + \rho_v\, o_\ell)\, H$.

All four witness components are quantities the prover already knows. A single DotProdPf then simultaneously verifies every round of the sumcheck.

4.3 Zero-Knowledge Polynomial Commitments

We still have one source of information leakage about the witness: the Quokka commitment and opening steps. Thus, in this part of the project you will fill in those gaps by changing Quokka slightly. At a high level, our modifications are twofold: first, we will change the commitment itself to be hiding; second, we will make the opening protocol zero-knowledge.

The zkQuokka.Pgen and zkQuokka.Commit procedures are given to the right, and the opening protocol is in Figure 4. The main change in Commit is the addition of a per-row blinding factor o_i, which is multiplied by the generator H and added to each row commitment. The opening protocol also changes significantly. In non-ZK Quokka, the prover simply sends the vector $\vec{b}^\top M$ to the verifier, who checks correctness by recomputing its MSM and checking equality to C'. In zkQuokka, both the prover and verifier compute C'; the prover then uses a zero-knowledge inner-product protocol (DotProdPf) to convince the verifier that the inner product of $\vec{a}$ and the vector committed under C' equals the value committed under C_v.

zkQuokka.Pgen(m):

$G_1, \ldots, G_m, H \leftarrow\!\!\$\ \mathbb{G}$

Return $(G_1, \ldots, G_m, H)$

zkQuokka.Commit(pp, P):

$M \leftarrow \mathsf{matrix}(P)$

For $i \in [m]$:

  $o_i \leftarrow\!\!\$\ \mathbb{F}$

  $C_i \leftarrow (\sum_{j=1}^{m} M_{i,j} G_j) + o_i H$

Return $(C_1, \ldots, C_m), (o_1, \ldots, o_m)$

4.4 Putting It All Together: zkDelphian

Combining all the components, we obtain zkDelphian, a zkSNARK for R1CS. Pseudocode of the interactive version of zkDelphian is in Figure 5. (If you read the section above on the Transcript struct it should be fairly clear how to apply Fiat-Shamir to the protocol in Figure 5; if you have questions, post on Piazza.)

The protocol is a fairly straightforward translation of the non-ZK version of Delphian from P2. We replace the non-ZK sumchecks with their ZK versions; we also use zkQuokka instead of the regular Quokka. Note that our zk sumcheck reduction works slightly differently in context: above we wrote the protocol as having the prover and verifier take as input a common commitment to the multivariate polynomial. In this case, however, we never explicitly materialize a commitment to all of h (or, indeed, to the input polynomials p_M in the second-phase sumchecks). We can get away with this because h and the p_Ms have special structure that the verifier can take advantage of to compute part of an evaluation (e.g., the value $e \leftarrow \widetilde{\mathsf{eq}}(r, r')$).

The only step that requires more careful explanation is the transition between the first and second phase. The first phase sumcheck ends with the verifier outputting the evaluation point $r' = (r'_1, \ldots, r'_{\log n})$ and a commitment $C_{h(r')}$ of a value claimed to be the evaluation of h at r'. Recall that in non-ZK Delphian, the verifier gets the claimed value $v_{h(r')}$ in the clear; then, the prover gives it values v_A, v_B, v_C claimed to be the decomposition of $v_{g_{res}(r')}$. (Recall also that the verifier can obtain $v_{g_{res}(r')}$ from $v_{h(r')}$ by simply dividing the latter value by $\widetilde{\mathsf{eq}}(r, r')$, which it can compute itself.) The verifier checks the claimed relationship between v_A, v_B, v_C and $v_{g_{res}(r')}$, then uses v_A, v_B, v_C as inputs for the second-phase sumchecks over the polynomials p_A, p_B, p_C respectively.

As discussed above, the main challenge in making this step ZK is that we cannot release v_A, v_B, v_C without revealing information about h and g_{res}. Thus, we apply the commit-and-prove methodology here as well: the prover sends Pedersen commitments to v_A, v_B, v_C then proves the claimed relationship holds under the commitments. With this, the verifier is convinced that the commitments it is given have the right relationship to $g_{res}(r')$ and are therefore the right inputs to the second-phase sumchecks.

5 Submission Instructions and Grading Criteria

To submit your project, first apply the changes to P1 and P2 described in Section 2 and then run the command

$\mathsf{zkQuokka.Open}[\mathcal{P}](\mathsf{pp}, P, \vec{C}, C_v, \vec{r}; \vec{o}, o_v)$	$\mathsf{zkQuokka.Open}[\mathcal{V}](\mathsf{pp}, \vec{C}, C_v, \vec{r})$
$C_1, \ldots, C_m \leftarrow \vec{C}; M \leftarrow \mathsf{matrix}(P)$	$C_1, \ldots, C_m \leftarrow \vec{C}$
$\vec{a} \leftarrow [\chi_j(\vec{r}_{\ell/2}, \ldots, \vec{r}_{\ell-1})]_{j\in[m]}$	$\vec{a} \leftarrow [\chi_j(\vec{r}_{\ell/2}, \ldots, \vec{r}_{\ell-1})]_{j\in[m]}$
$\vec{b} \leftarrow [\chi_k(\vec{r}_0, \ldots, \vec{r}_{\ell/2-1})]_{k\in[m]}$	$\vec{b} \leftarrow [\chi_k(\vec{r}_0, \ldots, \vec{r}_{\ell/2-1})]_{k\in[m]}$
$C' \leftarrow \sum_{k=1}^{m} \vec{b}_k C_k$	$C' \leftarrow \sum_{k=1}^{m} \vec{b}_k C_k$
$\vec{c} \leftarrow \vec{b}^\top M \ ; \ o_{\vec{c}} \leftarrow \langle \vec{b}, \vec{o} \rangle$	
..... Inner-product proof that C_v commits to $\langle \vec{c}, \vec{a} \rangle$	
$\vec{G}, G, H \leftarrow \mathsf{pp}$	$\vec{G}, G, H \leftarrow \mathsf{pp}$
$\vec{d} \leftarrow_{\$} \mathbb{Z}_p^m \ ; \ o_1, o_2 \leftarrow_{\$} \mathbb{Z}_p$	
$C_1 \leftarrow \mathsf{msm}(\vec{d}, \vec{G}) + o_1 H$	
$C_2 \leftarrow \langle \vec{d}, \vec{a} \rangle G + o_2 H$	
$\xrightarrow{C_1, C_2}$	
	$e \leftarrow_{\$} \mathbb{Z}_p$
$\xleftarrow{e}$	
$\vec{c}' \leftarrow e\vec{c} + \vec{d}$	
$o_{\vec{c}'} \leftarrow e o_{\vec{c}} + o_1 \ ; \ o_{v'} \leftarrow e o_v + o_2$	
$\xrightarrow{\vec{c}', o_{\vec{c}'}, o_{v'}}$	
	Return $eC' + C_1 = \mathsf{msm}(\vec{c}', \vec{G}) + o_{\vec{c}'} H$ $\wedge\ eC_v + C_2 = \langle \vec{c}', \vec{a} \rangle G + o_{v'} H$

Figure 4: The zkQuokka opening protocol. P is a multilinear polynomial in $n = m^2 = 2^\ell$ variables, and $\vec{r}$ is a vector of ℓ variables (ℓ is even). The procedure matrix takes a multilinear polynomial P and returns a matrix M of size $m \times m$ where $M_{j,k} = P_{jm+k}$ and P_{jm+k} is the coefficient of the monomial χ_{jm+k} in the evaluation form of P.

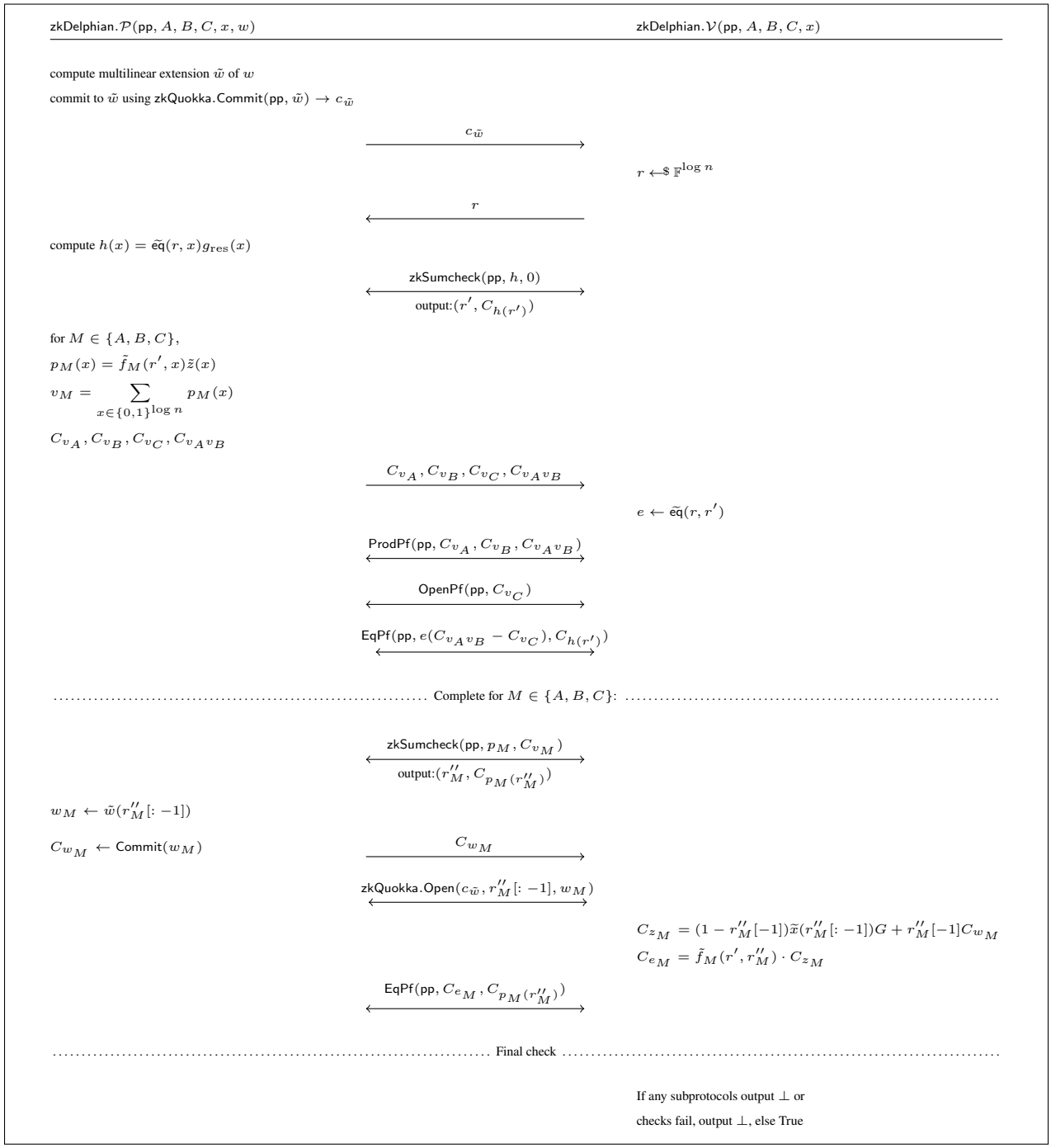


Figure 5: The zkDelphian zero-knowledge succinct argument of knowledge for R1CS. Transforming this into a zkSNARK using Fiat-Shamir is left as an exercise to the reader. The R1CS matrices A, B, C are $n \times n$ and $|x| = |w| = n/2$.

```
zip submission.zip -r p1/src/ p2/src p3/src -x '*/target/*'
```

from the directory containing your `p1/`, `p2/`, and `p3/` folders. Then, submit the resulting `submission.zip` file to autograder.io.

Autograder.io will run our test suite using your submitted code and compute your grade. Your grade will be the percentage of tests that pass. Any modifications you make to the `tests/` directory will not impact how your code is graded. (Feel free to add your own tests locally; this can be a good way to test your own understanding.)

Note that doing something "fishy" to try to get a better grade—for example, overwriting any equality methods—will result in failing the assignment.